

Module: Data Validation in Python

Importance · Range Checks · Cross-Field Rules · Consistency · Logging · Error Handling

1. Why Data Validation Matters

Definition

Data Validation is the process of checking whether data conforms to a defined set of rules, constraints, or expectations before it is used in computation, storage, or decision-making. A validation rule is any logical condition that data must satisfy to be considered correct, complete, and trustworthy.

In every real-world Python project — from ETL pipelines to ML feature engineering to REST API backends — raw data arrives dirty, inconsistent, or malformed. Without validation, these defects propagate silently and corrupt downstream systems, reports, and models.

1.1 Real-World Consequences of Skipping Validation

Domain	Invalid Data Example	Business Impact
E-commerce	Negative product quantity	Incorrect inventory, revenue loss
Banking / Finance	Loan amount = 0 or null	Approval of invalid applications
Healthcare	Age = -5 or 999	Wrong dosage computation, patient harm
Supply Chain	Delivery date < Order date	Broken logistics schedules
Machine Learning	Feature with 90% nulls	Model bias, silent accuracy drops

1.2 The Validation Pipeline — A Bird's Eye View

Every robust validation workflow follows four stages:

- Ingest — read raw data from file, API, database, or stream
- Validate — apply rules (range, cross-field, consistency, format)
- Log — record all invalid records with details for audit / debugging
- Handle errors — reject, quarantine, repair, or escalate based on severity

2. Range Checks

Definition

A Range Check verifies that a numeric or date field falls within an acceptable minimum and maximum boundary.

Range checks are the first line of defence against data entry errors, sensor malfunctions, and ETL glitches.

2.1 Basic Range Check

Let us start with the simplest form — validating a single value:

```
def check_range(value, field_name, min_val, max_val):
    """Return True if value is within [min_val, max_val], else False."""
    if value is None:
        print(f"[WARN] {field_name} is NULL - cannot validate range.")
        return False
    if not (min_val <= value <= max_val):
        print(f"[ERROR] {field_name} = {value} is outside [{min_val}, {max_val}]")
        return False
    return True

# — Test

records = [
    {"id": 1, "age": 28},
    {"id": 2, "age": -3},          # below minimum
    {"id": 3, "age": 152},        # above maximum (biologically impossible)
    {"id": 4, "age": None},       # missing
]
```

```

for r in records:
    is_valid = check_range(r["age"], "age", min_val=0,
                           max_val=120) print(f" Record {r['id']} → {'PASS' if is_valid
                           else 'FAIL'}")

# Output:
#   Record 1 → PASS
# [ERROR] age = -3 is outside [0, 120]
#   Record 2 → FAIL
# [ERROR] age = 152 is outside [0, 120]
#   Record 3 → FAIL
# [WARN] age is NULL - cannot validate range.
#   Record 4 → FAIL

```

2.2 Pandas-Based Range Check on a DataFrame

In production ETL, you validate entire columns at once using pandas:

```

import pandas as pd

df = pd.DataFrame({
    "order_id": [101, 102, 103, 104, 105],
    "quantity": [5, -1, 200, 0, 12],      # -1 is invalid; 200 might be
a cap
    "price": [10.5, 5.0, 0.0, 99.9, -3.0], # 0 and negative invalid
    "discount": [0.10, 0.50, 1.20, 0.05, 0.30], # must be 0.0 - 1.0
(0-100%)
})

rules = {
    "quantity": (1, 150),
    "price": (0.01, 99999),
    "discount": (0.0, 1.0),
}

issues = []

for col, (lo, hi) in rules.items():
    mask_out = ~df[col].between(lo, hi)      # rows that violate bad
    = df[mask_out][["order_id", col]].copy()
    bad["rule"] = f"{col} must be in [{lo}, {hi}]"
    issues.append(bad)

invalid_df = pd.concat(issues, ignore_index=True)
print(invalid_df.to_string(index=False))

# Output:
#   order_id  quantity      rule
#   102      -1      quantity must be in [1, 150]

```

```
# order_id price rule
# 103 0.0 price must be in [0.01, 99999]
# 105 -3.0 price must be in [0.01, 99999] #
order_id discount rule
# 103 1.20 discount must be in [0.0, 1.0]
```

2.3 Date Range Checks

Date fields are just as prone to range violations — ensure no future-dated birth records or pre-system dates:

```
from datetime import date

def validate_date_range(dt: date, field: str,
                        min_date: date, max_date: date) -> bool:
    if not (min_date <= dt <= max_date):
        print(f"[ERROR] {field} = {dt} out of range [{min_date}, {max_date}]")
        return False
    return True

today = date.today()
samples = [
    date(1985, 6, 15), # valid
    date(2035, 1, 1),  # future — probably wrong for date_of_birth
    date(1700, 3, 10), # too old — pre-system
]

for d in samples:
    validate_date_range(d, "date_of_birth",
                        min_date=date(1900, 1, 1),
                        max_date=today)
```

3. Cross-Field Validation Rules

Definition

Cross-Field Validation checks logical relationships between two or more fields in the same record.

These rules cannot be captured by looking at any single column in isolation — you need to compare fields against each other.

Example: end_date must always be greater than or equal to start_date.

3.1 Common Cross-Field Rule Patterns

Pattern	Example Rule	Domain
Date ordering	end_date >= start_date	Projects, bookings, HR
Amount vs limit	loan_amount <= credit_limit	Banking, subscriptions
Conditional required	If is_married=True → spouse_name not null	Government forms, insurance
Total = sum of parts	tax + base == total	Invoices, payroll
Discount vs price	discounted_price <= original_price	E-commerce
Age vs date	age == today - date_of_birth (years)	Healthcare, census

3.2 Python Implementation

```
import pandas as pd
from datetime import datetime

orders = pd.DataFrame({
    "order_id":      [1001, 1002, 1003, 1004],
    "order_date":    ["2024-01-10", "2024-03-05", "2024-06-01",
"2024-07-20"],
    "delivery_date"  ["2024-01-15", "2024-03-01", "2024-06-01",
: "2024-08-10"],
    "base_price":    [500,  200,  300,  450],
    "discount_amount": [50,  250,  100,  30],    # row 1002 has discount
> base!
    "total_amount":  [450,  200, 190, 420],      # row 1003 total is
wrong
    "is_express":    [True, False, True, False],
    "express_fee":    [30,   0,   0,  15],      # row 1003 is_express
but fee=0
})

# Convert dates
for col in ["order_date", "delivery_date"]:
    orders[col] = pd.to_datetime(orders[col])

violations = []

def record_violation(order_id, rule, detail):
```

```
violations.append({"order_id": order_id, "rule": rule,
                  "detail": detail})

for _, row in orders.iterrows():
    oid = row["order_id"]

    # Rule 1: delivery_date >= order_date
    if row["delivery_date"] < row["order_date"]:
        record_violation(oid, "date_order",
                        f"delivery {row['delivery_date'].date()} < order {row['order_date'].date()}")

    # Rule 2: discount must not exceed base price
    if row["discount_amount"] > row["base_price"]:
        record_violation(oid, "discount_cap",
                        f"discount={row['discount_amount']} > base={row['base_price']}")

    # Rule 3: total = base - discount
    expected_total = row["base_price"] - row["discount_amount"]
    if abs(row["total_amount"] - expected_total) > 0.01:
        record_violation(oid, "total_mismatch",
                        f"total={row['total_amount']} expected={expected_total}")

    # Rule 4: express orders must have express_fee > 0
    if row["is_express"] and row["express_fee"] <= 0:
        record_violation(oid, "express_fee_missing",
                        f"is_express=True but express_fee={row['express_fee']}")

viol_df = pd.DataFrame(violations)
print(viol_df.to_string(index=False))
```

4. Consistency Rules

Definition

Consistency Rules ensure that values are coherent across time, across related tables, or across different representations of the same entity.

Unlike range checks (single field) or cross-field rules (within one record), consistency rules often compare a record against external reference data — lookup tables, master data, or historical snapshots.

4.1 Types of Consistency Validation

- Referential Consistency — FK values must exist in a reference/master table

- Format Consistency — phone number, email, PAN/GSTIN always follow a pattern
- Enum Consistency — category fields must belong to a known set of valid values
- Temporal Consistency — values should not contradict known history (e.g., status from ACTIVE → DELETED should not revert to ACTIVE)
- Cross-dataset Consistency — same entity has matching values in two source systems

4.2 Referential and Enum Consistency

```
import pandas as pd
import re

# — Master / reference data
valid_department = {"HR", "Engineering", "Finance", "Marketing",
                    "Operations"}
valid_employment = {"Full-Time", "Part-Time", "Contract", "Intern"}
valid_country_codes = {"IN", "US", "GB", "DE", "SG", "AU"}

employees = pd.DataFrame({
    "emp_id": [1, 2, 3, 4, 5],
    "email": ["alice@corp.com", "bob@@corp.com", "carol@corp.com",
             "dave.corp.com", "eve@corp.com"],
    "dept": ["Engineering", "HR", "XYZ_Dept", "Finance", "IT"],
    "emp_type": ["Full-Time", "Contract", "Intern", "Freelancer",
                "Part-Time"],
    "country": ["IN", "US", "ZZ", "GB", "XX"],
    "pan_number": ["ABCDE1234F", "INVALID001", "PQRST5678G", "12345ABCDE",
                  "LMNOP9012Q"],
})

EMAIL_RE = re.compile(r"^[w.-]+@[w.-]+.w(2,)$")
PAN_RE = re.compile(r"^[A-Z]{5}[0-9]{4}[A-Z]$" # India PAN format

errors = []

for _, row in employees.iterrows():
    eid = row["emp_id"]

    # Email format
    if not EMAIL_RE.match(str(row["email"])):
        errors.append((eid, "invalid_email", row["email"]))

    # Department must be in approved set
    if row["dept"] not in valid_department:
        errors.append((eid, "invalid_dept", row["dept"]))

    # Employment type must be in approved set if
    row["emp_type"] not in valid_employment:
        errors.append((eid, "invalid_emp_type", row["emp_type"]))
```



```
# Country code must be recognised
if row["country"] not in valid_country_codes:
    errors.append((eid, "invalid_country", row["country"]))

# PAN format (only validate Indian employees)
if row["country"] == "IN" and not PAN_RE.match(str(row["pan_number"])):
    errors.append((eid, "invalid_pan", row["pan_number"]))

err_df = pd.DataFrame(errors, columns=["emp_id", "violation_type",
"bad_value"])
print(err_df.to_string(index=False))
```

4.3 Temporal Consistency — Tracking State Transitions

```
# Valid loan status transitions (state machine)
VALID_TRANSITIONS = {
    "APPLIED": {"APPROVED", "REJECTED"},
    "APPROVED": {"DISBURSED", "CANCELLED"},
    "DISBURSED": {"CLOSED"},
    "REJECTED": set(),
    "CANCELLED": set(),
    "CLOSED": set(),
}

loan_history = [
    {"loan_id": "L001", "from_status": "APPLIED", "to_status":
"APPROVED"}, # PASS
    {"loan_id": "L002", "from_status": "APPLIED", "to_status": "APPLIED"},
"APPROVED", # FAIL (reversal)
    {"loan_id": "L003", "from_status": "CLOSED", "to_status":
"DISBURSED"}, # FAIL
    {"loan_id": "L004", "from_status": "DISBURSED", "to_status": "CLOSED"},
# PASS
]

for rec in loan_history:
    allowed = VALID_TRANSITIONS.get(rec["from_status"], set())
    if rec["to_status"] not in allowed:
        print(f"[ERROR] Loan {rec['loan_id']}: "
              f"illegal transition {rec['from_status']}
              →
              {rec['to_status']}")
    else:
        print(f"[OK] Loan {rec['loan_id']}: "
              f"{rec['from_status']} → {rec['to_status']}")
```

5. Logging Invalid Records

Definition

Validation Logging is the practice of systematically recording every data quality issue — which record failed, which rule was violated, and when — so that errors can be audited, diagnosed, and corrected.

Good logs transform invisible data quality problems into actionable, traceable intelligence.

5.1 Why Logging Matters

- Auditable trail — prove to stakeholders that validation ran and what it caught
- Debugging — quickly pinpoint which records need manual review or reprocessing
- Monitoring — track error rates over time; spike = upstream change
- Compliance — GDPR, SOX, HIPAA all require data quality evidence

5.2 Python logging Module — The Right Way

```
import logging
import json
from datetime import datetime

# — Configure a structured logger
-----
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)-8s | %(message)s",
    datefmt="%Y-%m-%dT%H:%M:%S",
    handlers=[
        logging.StreamHandler(),    # console
        logging.FileHandler("validation_errors.log"), # file
    ]
)
logger = logging.getLogger("data_validator")

def log_invalid(record_id, field, rule, actual_value, expected=None):
    """Emit a structured JSON log line for every invalid record."""
    payload = {
        "ts":    datetime.utcnow().isoformat(),
        "record_id": record_id,
        "field": field,
        "rule": rule,
        "actual_value":
            actual_value, "expected":
            expected,
    }
    logger.warning(json.dumps(payload))
```

```
# — Usage example

records = [
    {"id": "R001", "age": 28, "salary": 50000},
    {"id": "R002", "age": -5, "salary": 120000},
    {"id": "R003", "age": 35, "salary": -1000},
]

for r in records:
    if not (0 <= r["age"] <= 120):
        log_invalid(r["id"], "age", "range[0,120]", r["age"])
    if r["salary"] < 0:
        log_invalid(r["id"], "salary", "non_negative", r["salary"],
expected=">=0")
```

5.3 Structured CSV Error Report

For business stakeholders who will review errors in Excel, write a clean CSV alongside your log:

```
import csv
from io import StringIO

class ValidationReport:
    def __init__(self):
        self.errors = []

    def add(self, record_id, field, rule, value, severity="ERROR"):
        self.errors.append({
            "timestamp": datetime.utcnow().isoformat(),
            "record_id": record_id,
            "field": field,
            "rule": rule,
            "bad_value": value,
            "severity": severity,
        })

    def to_csv(self, filepath):
        if not self.errors:
            print("No validation errors – report not written.")
            return
        fieldnames =
["timestamp", "record_id", "field", "rule", "bad_value", "severity"]
        with open(filepath, "w", newline="") as f:
            w = csv.DictWriter(f, fieldnames=fieldnames)
            w.writeheader()
            w.writerows(self.errors)
        print(f"Validation report saved → {filepath} ({len(self.errors)}
issues)")
```

```
def summary(self):
    total = len(self.errors)
    errors = sum(1 for e in self.errors if e["severity"] == "ERROR")
    warns = sum(1 for e in self.errors if e["severity"] == "WARN")
    print(f"\n{'='*50}")
    print(f"Validation Summary: {total} issues | {errors} ERRORS | {warns} WARNs")
    print(f"{'='*50}\n")

# — Usage

report = ValidationReport()
report.add("R002", "age", "range[0,120]", -5)
report.add("R003", "salary", "non_negative", -1000)
report.add("R004", "email", "format", "invalid@@", severity="WARN")
report.summary()
report.to_csv("validation_report.csv")
```

6. Error Handling Patterns

Definition

An Error Handling Pattern is a strategy that defines what happens to invalid records after they are detected.

Choosing the right pattern depends on your business requirements: how critical is data completeness vs data accuracy?

6.1 The Four Core Patterns

Pattern	Description	When to Use
Fail-Fast	Stop processing immediately on first error	Critical pipelines where any bad record invalidates the whole batch
Reject & Continue	Skip invalid records; continue processing valid ones	High-volume feeds; partial results are acceptable
Quarantine	Route bad records to a separate error store for later review	Production ETL; human or automated review before retry
Auto-Repair	Apply correction rules to fix common issues automatically	Known, predictable errors with deterministic fixes

6.2 Pattern 1 — Fail-Fast

```
class ValidationError(Exception):
    """Raised when a critical validation rule is
    violated."""
    def __init__(self, record_id, field,
        message):
        self.record_id =
        record_id
        self.field =
        field
        super().__init__(f"Record {record_id} | Field '{field}': {message}")

def process_critical_payment(payment: dict):
    """Fail immediately if any rule is broken – no partial
    processing."""
    if payment["amount"] <= 0:
        raise ValidationError(payment["id"], "amount",
            f"Must be positive, got {payment['amount']}")
    if payment["currency"] not in {"INR", "USD", "EUR"}:
        raise ValidationError(payment["id"], "currency",
            f"Unsupported currency:
{payment['currency']}")
    # ... proceed with processing
    print(f"Payment {payment['id']} processed: {payment['currency']}
{payment['amount']}")

payments = [
    {"id": "P001", "amount": 5000, "currency": "INR"},
    {"id": "P002", "amount": -200, "currency": "USD"},      # will fail
]

for p in payments:
    try:
        process_critical_payment(p)
    except ValidationError as ve:
        print(f"[FAIL-FAST] {ve}")
        break # halt entire batch
```

6.3 Pattern 2 — Reject & Continue

```
def validate_and_split(records: list) -> tuple:
    """
    Separate records into valid and invalid without stopping.
    Returns (valid_records, invalid_records).
    """
    valid, invalid = [], []
    for r in records:
        errors = []
        if r.get("age") is None or not (0 <= r["age"] <= 120):
            errors.append(f"age out of range: {r.get('age')}")
        if r.get("score") is None or not (0.0 <= r["score"] <= 100.0):
            errors.append(f"score out of range: {r.get('score')}")
        if errors:
```

```

        r["_errors"] = errors
        invalid.append(r)
    else:
        valid.append(r)
    return valid, invalid

data = [
    {"id":1, "age":25, "score":88.5},
    {"id":2, "age":-1, "score":72.0}, # bad age
    {"id":3, "age":40, "score":105.0}, # bad score
    {"id":4, "age":33, "score":91.3},
]

good, bad = validate_and_split(data)
print(f"Valid: {len(good)} | Rejected: {len(bad)}")
for b in bad:
    print(f"    ID={b['id']} → {b['_errors']}")

```

6.4 Pattern 3 — Quarantine Store

```

import pandas as pd
from datetime import datetime

class QuarantineStore:
    """Accumulate invalid records and flush to CSV / database."""

    def _init_(self):
        self._records = []

    def quarantine(self, record: dict, reason: str, pipeline: str = "default"):
        entry = {
            "quarantine_ts": datetime.utcnow().isoformat(),
            "pipeline": pipeline,
            "reason": reason,
        }
        entry.update(record)
        self._records.append(entry)

    def flush_to_csv(self, path: str):
        df = pd.DataFrame(self._records)
        df.to_csv(path, index=False)
        print(f"Quarantined {len(df)} records → {path}")
        self._records.clear()

    def _len_(self):
        return len(self._records)

```

```
# — Demo

store = QuarantineStore()

employee_batch = [
    {"emp_id": "E01", "salary": 80000, "dept": "Engineering"},
    {"emp_id": "E02", "salary": -500, "dept": "Finance"}, # quarantine
    {"emp_id": "E03", "salary": 60000, "dept": "Unknown_Dept"}, # quarantine
]
valid_depts = {"HR", "Engineering", "Finance", "Marketing"}
for emp in employee_batch:
    if emp["salary"] < 0:
        store.quarantine(emp, "negative_salary", pipeline="hr_ingestion")
    elif emp["dept"] not in valid_depts:
        store.quarantine(emp, "invalid_department", pipeline="hr_ingestion")
    else:
        print(f"Processing {emp['emp_id']} ...")

store.flush_to_csv("quarantine_hr.csv")
```

6.5 Pattern 4 — Auto-Repair

```
def auto_repair(record: dict) -> dict:
    """
    Apply deterministic fixes for known, common data quality issues.
    Always log what was changed for full transparency.
    """
    original = record.copy()
    repairs = []

    # Fix 1: Strip extra whitespace from string fields
    for key, val in record.items():
        if isinstance(val, str) and val != val.strip():
            record[key] = val.strip()
            repairs.append(f"{key}: whitespace stripped")

    # Fix 2: Negative quantity → set to 0 (cannot ship negative items)
    if record.get("quantity", 0) < 0:
        record["quantity"] = 0
        repairs.append(f"quantity: {original['quantity']} → 0 (floored)")

    # Fix 3: Uppercase country codes
    if "country" in record and isinstance(record["country"], str):
        upper = record["country"].upper()
        if upper != record["country"]:
            record["country"] = upper
            repairs.append(f"country: '{original['country']}' → '{upper}'")
```

```
    if repairs:
        print(f"[AUTO-REPAIR] Record {record.get('id')}: {'',
'.join(repairs)}")
    return record

# — Test
```

```
raw_records = [
    {"id": 1, "name": " Alice ", "quantity": 5, "country": "in"},
    {"id": 2, "name": "Bob", "quantity": -3, "country": "US"},
    {"id": 3, "name": "Carol", "quantity": 8, "country": "gb"},
]

cleaned = [auto_repair(r) for r in raw_records]
for c in cleaned:
    print(c)
```

7. Putting It All Together — Production Validation Pipeline

The following end-to-end example combines all concepts: a class-based validator for an employee HR dataset that performs range checks, cross-field rules, consistency validation, logging, and applies the quarantine + reject pattern.

```
import pandas as pd
import logging
import csv
from datetime import datetime, date

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)-8s | %(message)s",
    handlers=[logging.StreamHandler(),
logging.FileHandler("hr_pipeline.log")]
)
logger = logging.getLogger("hr_validator")

VALID_DEPTS = {"HR", "Engineering", "Finance", "Operations", "Marketing"}
VALID_GRADES = {"L1", "L2", "L3", "L4", "L5", "L6"}
RETIRE_AGE = 60
MIN_HIRE_YEAR = 1980

class HRValidator:
    def __init__(self):
        self.valid_rows = []
```



```

        self.quarantine      =
        [] self._report_rows =
        []

    def _flag(self, row, rule, detail, severity="ERROR"):
        logger.warning(f"[{severity}] EmpID={row['emp_id']} | rule={rule} | {detail}")
        self._report_rows.append({
            "ts":
                datetime.utcnow().isoformat(), "emp_id":
                    row["emp_id"],
            "rule":      rule,
            "detail":    detail,
            "severity":  severity,
        })

    def validate(self, df: pd.DataFrame) -> pd.DataFrame:
        logger.info(f"Validating {len(df)} records ...")

        for _, row in df.iterrows():
            violations = []

            # — Range checks
            if not (18 <= row["age"] <= RETIRE_AGE):
                self._flag(row, "age_range",
                    f"age={row['age']} not in [18,{RETIRE_AGE}]")
                violations.append("age_range")

            if
                ro
                w[
                "s
                al
                ar
                y"
                ]
                <
                15
                00
                0:
                se
                lf
                .
                fl
                ag
                (r
                ow
                ,
                "s
                al
                ar
                y_
                mi
                ni

```



```

        grade_min = expected_min_salary.get(row["grade"],
0) if row["salary"] < grade_min:
    self._flag(row, "salary_grade_mismatch",
        f"Grade {row['grade']} min={grade_min}
        but
salary={row['salary']}")
    violations.append("salary_grade")

# — Route record
-----
- if violations:
    self.quarantine.append(row.to_dict())
else:
    self.valid_rows.append(row.to_dict())

logger.info(f"Done: {len(self.valid_rows)} valid | "
            f"{len(self.quarantine)} quarantined.")
return pd.DataFrame(self.valid_rows)

def save_report(self, path="validation_report.csv"):
    pd.DataFrame(self._report_rows).to_csv(path, index=False)
    logger.info(f"Validation report → {path}")

# — Run
-----
df = pd.DataFrame({
    "emp_id":
        ["E01", "E02", "E03", "E04", "E05"], "age":  [30,
            -1,    45,    62,    28],
    "salary":  [50000, 30000, 10000, 95000, 45000],
    "department": ["Engineering", "HR", "Unknown", "Finance", "Marketing"],
    "grade":  ["L3", "L2", "L2", "L5", "L2"],
    "join_date":
        ["2015-06-01", "2020-01-15", "2023-03-01", "1975-08-10", "2021-09-01"],
})

validator = HRValidator()
clean_df = validator.validate(df)
validator.save_report()
print(f"\nClean records ready for
downstream:\n{clean_df[['emp_id', 'age', 'salary']].to_string(index=False)}")

```

8. Quick Reference — Cheat Sheet

Concept	Python Idiom	Key Library / Function
Range check	value between(lo, hi)	pandas .between() / plain comparison
Null check	pd.isnull(val) or val is None	pandas .isnull() / .notna()

Regex format	<code>re.match(pattern, value)</code>	<code>import re</code>
Enum check	<code>value in valid_set</code>	Python set — O(1) lookup
Cross-field	<code>row['end'] >= row['start']</code>	pandas iterrows / vectorised mask
Logging	<code>logger.warning(json.dumps(...))</code>	<code>import logging + json</code>
Error report	<code>pd.DataFrame(errors).to_csv()</code>	pandas + csv module
Fail-Fast	<code>raise ValidationError(...)</code>	custom Exception subclass
Quarantine	<code>bad_records.append(row)</code>	list + DataFrame + <code>to_csv()</code>
Auto-repair	<code>record[field] = fixed_value</code>	plain Python dict mutation
Schema validation	<code>schema.validate(df)</code>	pandera / pydantic / great_expectations

Pro Tip — Advanced Libraries to Explore

pandera — schema-based DataFrame validation with type annotations and statistical checks.

pydantic — model-level validation for APIs and config objects using Python type hints.

great_expectations — enterprise-grade data quality suites with HTML reports and CI integration.

cerberus — lightweight, schema-driven validation for Python dicts.